

Лабораторная работа №1.
Основы Linux

Цели лабораторной работы

1. Изучить базовое устройство Linux;
2. Научиться взаимодействовать с Linux.

Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Выполнить базовые операции с Linux.

Методические материалы

Что такое Linux?

Linux — это UNIX-подобная операционная система, с открытым исходным кодом, разработанная финским студентом Линусом Торвальдсом как альтернатива учебной UNIX-подобной операционной системе Minix. Термин **UNIX-подобный** обозначает наследование базовых идей и архитектурных решений оригинальной системы UNIX. В настоящее время выпущены стандарты **POSIX** (Portable Operating System Interface) и **SUS** (Single UNIX Specification). **POSIX** определяет API операционной системы, что призвано обеспечить переносимость программ на уровне исходного кода между различными UNIX-подобными системами. Системы, реализующие POSIX называются POSIX-совместимыми. **SUS** определяет соответствие реализации интерфейса операционной системы и позволяет использовать торговую марку **UNIX**.

На базе Linux выпущено множество различных дистрибутивов, которые составляют семейство Linux-подобных операционных систем или просто Linux систем.

Структурно Linux состоит из ядра и пользовательского пространства.

Ядро Linux

Ядро Linux — центральный компонент системы, обеспечивающий взаимодействие между аппаратными средствами и приложениями. Для уведомления ядра о событиях используются аппаратные и программные прерывания:

- *Аппаратные прерывания* вызываются со стороны и в сторону аппаратных компонентов, таких как: мыши, клавиатуры и т.д. Данный тип прерываний требует высоких привилегий и доступен только ядру.
- *Программные прерывания* вызываются со стороны и в сторону пользовательских программ, для обращения к компонентам ядра. Данный тип прерываний используется для реализации системных вызовов.

Ядро Linux является монолитным и выполняет следующие функции:

- **Управление памятью** — Выделение памяти под процессы, отслеживание того, каким образом и сколько памяти используется, изоляция памяти;
- **Управление процессами** — Управление процессами, определение того, какие процессы и как могут использовать центральный процессор (ЦП);
- **Драйверы устройств** — Действуют в качестве посредников между оборудованием и приложениями/процессами.
- **Системные вызовы и безопасность** — получение запросов на обслуживание от процессов приложений.

Плюсы монолитного ядра:

- Высокая производительность. Прямое взаимодействие компонентов без накладных расходов на межпроцессное взаимодействие (IPC) и переключение контекста обеспечивает быстрый обмен данными и обработку системных вызовов.
- Низкие задержки. Обработка прерываний и вызовов происходит максимально оперативно.
- Простота разработки (в определённых аспектах). Разработчику не нужно проектировать сложные интерфейсы для взаимодействия между сервисами ядра, так как всё работает в одном адресном пространстве.
- Эффективное использование ресурсов. Единое адресное пространство улучшает локальность данных и инструкций, что положительно сказывается на производительности кэша процессора.

Минусы монолитного ядра:

- Низкая отказоустойчивость. Сбой в любом компоненте ядра (например, в драйвере устройства) может привести к аварийному завершению всей системы.
- Сложность отладки. Из-за тесной связи компонентов сложно локализовать и исправить ошибку.
- Сложность поддержки и обновления. Большая и сложная кодовая база затрудняет её понимание, модификацию и верификацию.
- Проблемы с безопасностью. Поскольку все компоненты работают с максимальными привилегиями и имеют прямой доступ к памяти и аппаратуре, уязвимость в любом из них может быть использована

злоумышленником для компрометации всей системы. Отсутствие изоляции между компонентами — ключевой риск.

- Ограниченная масштабируемость. Архитектура сложнее адаптируется к распределённым системам или системам с множеством разнородных процессоров.

Пользовательское пространство

Пользовательское пространство Linux — пространство, где существуют и работают пользовательские процессы. Данные процессы изолированы друг от друга и ограничены в прямом доступе к аппаратным компонентам. Эти ограничения необходимы для обеспечения стабильной работы как самих процессов, так и ядра.

Где используется Linux?

Операционная система Linux используется в самых разных системах и средах:

- **Суперкомпьютеры.** Все 500 машин из списка самых мощных суперкомпьютеров в мире работают под управлением Linux;
- **Веб-серверы.** Linux используется практически на всех серверах в Сети. Согласно опросам агентства W3Techs, 82% веб-сайтов развёрнуты на машинах с установленным Linux- и Unix-системами;
- **Смартфоны.** Операционная система Android, которая установлена на большинстве смартфонов в мире, работает на ядре Linux;
- **Роутеры.** Операционные системы, установленные в Wi-Fi-маршрутизаторах, также основаны на ядре Linux;
- **Умный дом.** Linux используют многие устройства в умных домах: холодильники, телевизоры, стиральные машины;
- **Авиация и транспорт.** Linux установлена на бортовых компьютерах автомобилей Tesla и в машинах с автопилотом от Google. Ещё Linux используется в авиации: большинство американских систем для отслеживания трафика самолётов разработано на базе Linux.

Файловая система Linux

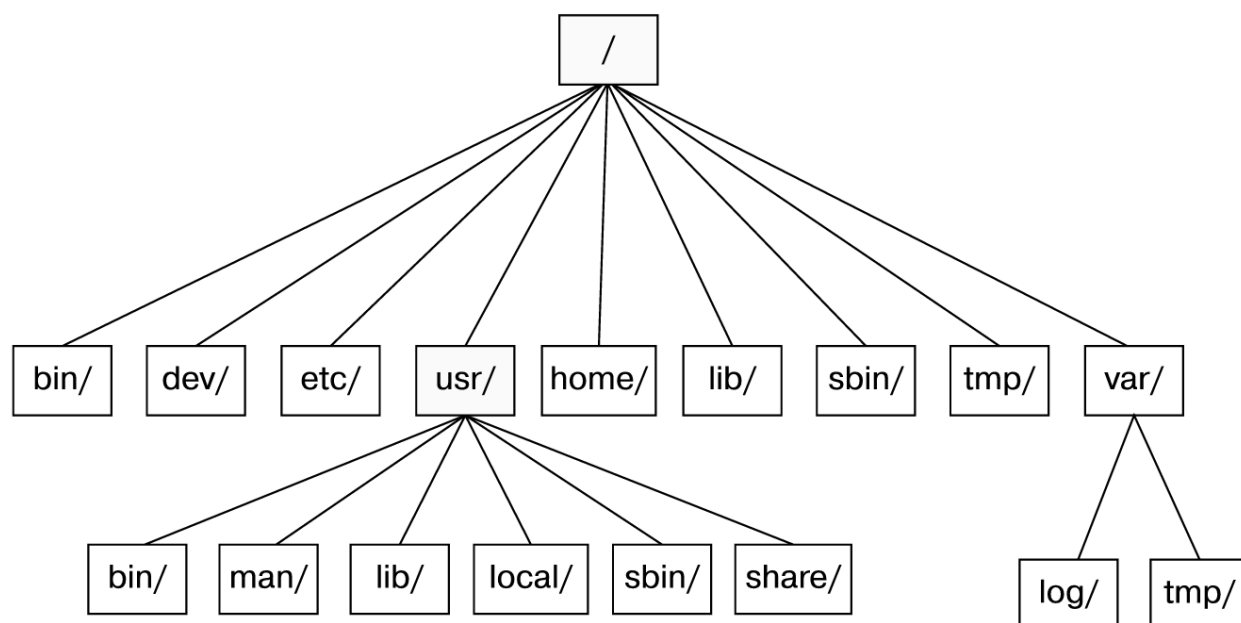
Файловая система — это некий упорядоченный способ учета, хранения и извлечения данных на различных носителях. Например, жестких дисках, SSD, USB-флешках, сетевых устройствах.

Структурирование файловой системы охватывает процесс форматирования носителя и определения логики размещения файлов и папок на нем. В итоге она

должна обеспечивать удобное размещение данных, их быстрый поиск и высокий уровень надежности.

Контроль за файлами и папками осуществляется операционной системой, что требует их взаимной совместимости и корректной работы в рамках данной ОС.

Файловая система Linux организована в форме единой иерархической структуры, которую образно можно представить в виде дерева. Она содержит в себе каталоги и подкаталоги, образуя таким образом вложенную структуру. Все файлы и каталоги начинаются от корневого каталога, который обозначается символом `/`, а далее распределяются по отдельным ветвям и листьям этого дерева. Таким образом создается логическая и удобная организация данных. Важно помнить, что несмотря на единую структуру, фактически данные могут располагаться на различных устройствах, в том числе и в сети. На рисунке ниже представлена схематичная структура каталогов Linux.



Linux придерживается стандартов иерархии файловой системы (Filesystem Hierarchy Standard, FHS). Эти стандарты определяют основные правила организации и содержания каталогов в системах, аналогичных UNIX. Они обеспечивают единообразие в структуре файловой системы, делая работу с разными дистрибутивами более предсказуемой и понятной для пользователей и разработчиков.

Также существуют специальные файлы, предназначенные для особых случаев, например, файлы устройств. Данные файлы реализуют возможность общения между процессами.

Наиболее важные подкаталоги:

- `/bin` — содержит готовые к запуску программы (также известные как исполняемые файлы), включая большинство основных команд Unix, таких, как `ls` и `cp` (Согласно FHS, данная папка содержит минимально необходимые программы для запуска ОС, если папка `/usr/bin` подключается после загрузки ОС);
- `/dev` — содержит файлы устройств;
- `/etc` — центральный каталог конфигурации системы. Содержит пароль пользователя, загрузочные файлы, файлы устройств, сетевые настройки и др;
- `/home` — содержит домашние (личные) каталоги для обычных пользователей;
- `/lib` — в этом каталоге находятся файлы библиотек с кодом, который могут использовать исполняемые файлы. Существует два типа библиотек: статические и разделяемые. Каталог `/lib` должен содержать только разделяемые библиотеки, но другие каталоги `lib`, такие, как `/usr/lib`, включают обе разновидности, а также другие вспомогательные файлы;
- `/proc` — предоставляет системную статистику через доступный для просмотра интерфейс каталогов и файлов. Каталог `/proc` содержит информацию о запущенных в данный момент процессах, а также некоторые параметры ядра. Файлы в данных каталогах также являются специальными;
- `/run` — содержит данные времени выполнения, относящиеся к системе, включая определенные идентификаторы процессов, файлы сокетов, записи состояния и во многих случаях системный журнал;
- `/sys` — похож на каталог `/proc` тем, что он предоставляет интерфейс устройствам и системе;
- `/sbin` — место для системных исполняемых файлов. Программы в каталогах `/sbin` связаны с управлением системой, поэтому простые пользователи обычно не имеют компонентов `/sbin` в своих путях команд. Многие из утилит в этом каталоге работают, только если запущены от имени суперпользователя (Согласно FHS, данная папка содержит минимально необходимые программы для запуска ОС, если папка `/usr/sbin` подключается после загрузки ОС);
- `/tmp` — место для хранения небольших, временных, не особо важных файлов. Любой пользователь может читать из каталога `/tmp` и записывать в него, но у пользователя может не быть доступа к файлам другого пользователя. Многие программы задействуют этот каталог в качестве рабочей области. Если какой-то файл важен, не помещайте его в каталог `/tmp`, потому что большинство дистрибутивов очищают его при загрузке, а некоторые даже периодически удаляют старые файлы. Также в некоторых случаях данный каталог может размещаться в оперативной памяти вместо диска;
- `/usr` — сокращение от `user` (пользователь), однако в этом подкаталоге нет пользовательских файлов. Вместо этого он содержит большую иерархию каталогов, включая основную часть системы Linux. Многие имена каталогов в

/usr совпадают с именами в корневом каталоге (например, /usr/bin и /usr/lib), и они содержат файлы одного типа. (Согласно FHS, данная папка может подключаться после загрузки ОС);

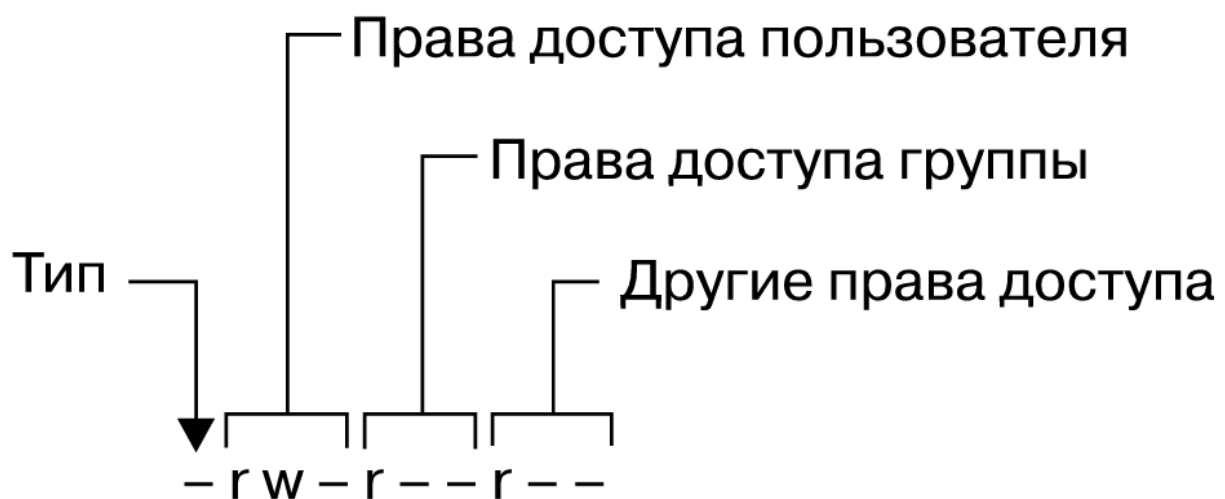
- /var — подкаталог переменных, куда программы записывают информацию, которая может изменяться с течением времени. Здесь находятся системные журналы, отслеживание активности пользователей, кэши и другие файлы, создаваемые системными программами и управляемые ими. (Здесь также есть каталог /var/tmp, но система не стирает его при загрузке.)

Режимы файлов и права доступа

Каждый файл Linux имеет набор прав доступа, которые определяют, можете ли вы читать, записывать или запускать файл. Команда `ls -l` отображает права доступа:

```
ls -l
-rw-r--r-- 1 someuser somegroup 530 Nov 27 23:56 somefile.txt
```

Режим файла `-rw-r--r--` представляет права доступа к файлу и некоторую дополнительную информацию.



Первый символ режима — это тип файла:

- `-` — Обычный файл, может содержать текстовые или бинарные данные
- `c` — Специальный файл с прямым небуферизованным доступом. Может требовать выравнивания данных для чтения и записи.
- `b` — Специальный блочный файл с буферизованным доступом. Не требует выравнивания, имеется задержка для чтения/записи.
- `l` — Символическая ссылка.
- `d` — Директория.

Важно: `ls -l` не всегда может точно определить тип файла. Например, папка `/proc` состоит исключительно из специальных файлов, однако их тип — `-`.

Оставшаяся часть режима файла содержит права доступа, которые разбиваются на три равных набора по три позиции: пользователь (user), группа (group) и остальные (other). Каждый набор соответствует формату `rwX`, где вместо буквы может стоять `-`:

- `r` — файл доступен для чтения;
- `w` — файл доступен для записи;
- `x` — файл является исполняемым;
- `-` — означает «ничего», то есть права нет.

Существуют также специальные биты расширения прав: SUID, SGID, StickyBit.

Права доступа пользователя (первый набор) принадлежат пользователю — владельцу файла. В примере это пользователь `someuser`. Второй набор — права доступа группы — предназначен для группы файла (в примере, для группы `somegroup`). Любой пользователь, относящийся к данной группе, может воспользоваться этими правами. Все остальные в системе имеют доступ в соответствии с третьим набором — правами доступа всех остальных. (Иногда их называют правами доступа world (то, что доступно всему миру)).

За блоком прав следуют:

- Количество жестких ссылок
- UID
- GID
- Размер файла
- Дата последнего изменения
- Имя файла

TTY. Подсистема общения с ОС Linux

TTY (teletype, телетайп) — исторически данный термин описывал класс устройств, позволяющих обмениваться текстовыми сообщениями. Их появление связано с развитием телеграфной связи. В современных ОС, таких, как Linux, данный термин обозначает специальный драйвер, отвечающий за обработку ввода/вывода системы через стандартные средства:

- Телетайп (физическое устройство, устарел);

- Видеотерминал (физическое устройство, может встречаться на старых системах);
- Виртуальный терминал (подсистема `tty`, часть ядра);
- Псевдотерминал (драйвер `pty`, эмулятор терминала находится в пользовательском пространстве).

Подсистема `tty` представляется для процесса в виде трёх специальных файлов:

- `stdin` — входящая информация;
- `stdout` — исходящая информация;
- `stderr` — сообщения об ошибках.

Любой процесс в системе, запущенный из `tty` сессии, имеет к ней привязку и называется интерактивным.

Подсистема `tty` отвечает не только за передачу ввода/вывода, но и за `Line Discipline`.

`Line Discipline` отвечает за преобразование `ctrl` кодов в сигналы процессу и накопление ввода до нажатия клавиши Ввод (`Enter`). Это называется `Canonical mode`. Если приложению необходимо отключить `Line Discipline`, `tty` переводится в `Raw mode`.

Обычно главным процессом в `tty` сессии является оболочка (`shell`).

Программа и процесс. Иерархия процессов

Программа — исполняемый файл, определяющий некоторую работу. Находится в файловой системе и может быть запущен.

Процесс — запущенный экземпляр программы. Для каждого процесса операционная система создаёт набор ресурсов:

- Виртуальная память — изолированная память со своим адресным пространством, с которой может взаимодействовать только текущий процесс. (Технически, существует огромное количество способов получить доступ к этой памяти из другого процесса, хотя бы в рамках процесса отладки);
- Таблица файловых дескрипторов — Таблица открытых процессом файлов, в том числе специальных (`stdin`, `stdout` и `stderr` находятся в этой таблице);
- Свойство `cwd` — `Current Work Directory`, текущий рабочий каталог. Директория, относительно которой определяются все относительные файловые пути;
- Свойство `env` — `Environment`, окружение. Набор пар ключ-значение, который процесс получает от родителя;

- Свойства UID и GID — Идентификатор пользователя и группы, от лица которых выполняется процесс. Данные поля влияют на проверку прав доступа к файлам;
- Свойство Код возврата — Код возврата это число в диапазоне 0-255, определяющая статус завершения процесса и выставляется обычно самой программой. 0 обозначает успех.

У каждого процесса, кроме корневого процесса ОС, имеется родительский процесс. Если родительский процесс завершается до дочернего, владение дочерним процессом передаётся вверх, вплоть до корневого процесса.

В Linux нельзя создать новый процесс с нуля, для создания нового процесса требуется клонировать текущий, а потом, если необходимо, заменить его программу. Важно помнить, что дочерний процесс наследует часть ресурсов родительского, а именно: таблицу файловых дескрипторов, переменные окружения, текущий каталог, владельца и группу, обработчики сигналов. Замена программы на новую не удаляет данные ресурсы (Обычно, именно эта разница в поведении между Linux и Windows (где ресурсы обычно не наследуются), приводит к огромному количеству ошибок и странному поведению при разработке кроссплатформенного ПО, особенно на языке python).

В контексте tty также существует дополнительное свойство процесса: фоновый процесс. Фоновый процесс отключается от канала ввода терминала, но продолжает свою работу и может выводить информацию на терминал.

Linux и SELinux

В настоящее время существует два типа Linux:

- Обычный Linux, на базе которого обычно создаются пользовательские дистрибутивы. Легок в обслуживании, в установке новых компонентов системы, но средства защиты специально снижены в пользу большего удобства. Система контроля доступа AppArmor. Основной принцип: разрешено всё, что не запрещено.
- Security-Enhanced Linux (SELinux), на базе которого обычно создают дистрибутивы с требованиями к повышенной безопасности. Отличается изначально мощной системой изоляции процессов, контроля сетевых функций и контроля доступа к ядру. Система контроля доступа SELinux. Основной принцип: запрещено всё, что не разрешено.

Механизмы дополнительной защиты работают поверх стандартной системы прав, расширяя её.

Пакетная система. Обновление ОС Linux.

Управление пакетами

Системы управления пакетами («менеджер пакетов» или «пакетный менеджер») - это набор программного обеспечения, позволяющего управлять процессом установки, удаления, настройки и обновления различных компонентов ПО.

Различные дистрибутивы ОС Linux имеют свои форматы пакетов:

- `.deb` — Debian и производные (Ubuntu, Mint и т.д.);
- `.rpm` — Red Hat и производные (CentOS, Fedora и т.д.), OpenSUSE;
- `.pkg.<постфикс-архиватора>` — Arch Linux и производные;
- `.apk` — Android;
- `.ebuild` — Gentoo.

Все системы управления пакетами имеют схожие команды. Разберём основные на примере `apt`. **APT** (Advanced Package Tool) является инструментом управления пакетами, который широко используется в дистрибутивах Debian и Ubuntu.

Основные команды `apt` :

- `apt update` — обновить список доступных пакетов из репозиториев;
- `apt upgrade` — обновить все установленные пакеты до последних версий;
- `apt dist-upgrade` — обновить все установленные пакеты до последних версий, удалить устаревшие, установить новые зависимости (Будьте аккуратны, если удалённый устаревший пакет неявно использовался в другой программе, она сломается);
- `apt install <package_name>` — установить или обновить указанный пакет из репозиториев;
- `apt remove <package_name>` — удалить указанный пакет, оставляя файлы конфигурации;
- `apt purge <package_name>` — удалить указанный пакет, удаляя все файлы конфигурации;
- `apt search <keyword>` — искать пакеты по ключевому слову в репозиториях;
- `apt show <package_name>` — показать подробную информацию о пакете.

Обновление ОС Linux

Обновление Linux можно разделить по уровням влияния:

- Обновление пользовательских пакетов — в результате вызова `apt update` && `apt upgrade` обновляются только пользовательские пакеты, не затрагивающие систему в целом. Такое обновление не требует перезагрузки

компьютера, и, как правило, зависимые программы автоматически перезапускаются.

- Обновление системных пакетов ядра — в результате вызова `apt update && apt upgrade` обновляются компоненты ядра. Такое обновление будет иметь результат только после перезагрузки системы, так как ядро загружается только один раз.
- Обновление дистрибутива — комплексное обновление, меняющее базовую версию дистрибутива. Для выполнения подобного обновления требуется наличие специального инструмента, либо чистая установка. Обычно такого рода обновление не требуется до завершения периода LTS.

Ход работы

Для выполнения данной лабораторной работы требуется Linux-подобная ОС.

Настройте окружение согласно одного из вариантов:

1. Основная ОС или Dual-boot, дистрибутив на базе Linux (Ubuntu и др.) или SELinux (Red Hat и др.).
 2. Компонент WSL2 для Windows 10/11 Pro
 3. MacOS (несмотря на допустимость этого варианта, MacOS развивалась в другой ветке UNIX-подобных систем (BSD), могут быть сложности с этим вариантом)
 4. Виртуальная система через гипервизор (например, Oracle VirtualBox)
-

Изучение стандартных программ

Изучите стандартный набор команд Linux, который часто используется. Внесите в отчёт назначение каждой команды и пример использования.

Изучите и опишите программы общего блока: `man`, `clear`, `history`, `whatis`, `which`, `sleep`.

Изучите и опишите программы работы с пользователями: `who`, `whoami`, `w`, `id`, `groups`, `passwd`, `su`, `sudo`, `login`, `adduser`, `useradd`, `usermod`, `userdel`, `addgroup`, `groupadd`, `groupmod`, `groupdel`.

Изучите и опишите программы для работы с файловой системой: `pwd`, `ls`, `cd`, `mkdir`, `rm`, `cp`, `mv`, `touch`, `cat`, `less`, `head`, `tail`, `grep`, `df`, `du`, `find`, `chmod`, `chown`, `chgrp`, `tar`.

Изучите и опишите программы для работы с процессами: `ps`, `top`, `kill`, `nice`, `free`.

Изучите и опишите программы для работы с сетью: `ip`, `ping`, `traceroute`.

Изучение файловой системы

Изучите и опишите, чем отличаются программы в папках: `/bin`, `/sbin`, `/usr/bin` и `/usr/sbin`.

Изучите и опишите структуру папки `/proc`.

Используя команду `ls -l` изучите типы файлов в директориях `/dev` и `/proc`, верно ли, что в директории `/proc` находятся обычные файлы?

Работа с пакетным менеджером

Используя пакетный менеджер найдите пакет `nano`, а потом установите его.

Используя ранее изученную команды общего блока, опишите, для чего `nano` предназначен и где находится его исполнимый файл.

Контрольные вопросы

1. Что такое Linux?
2. Где работают программы пользователя?
3. Основные характеристики файловой системы Linux?
4. Как строится иерархия процессов Linux?
5. Почему у `su` и `sudo` в блоке прав пользователя вместо `x` стоит `s`?
6. В чём разница между `su` и `login`?
7. В чём разница между Linux и SELinux?